

Aula 20 — Projeto Final e Apresentação

Módulo: Projeto, DevOps e Mercado

Carga horária: 4 horas

Projeto: Full Stack (.NET API + React ou Blazor + Docker)

Objetivo da aula:

Preparar o aluno para o **mercado de trabalho**, consolidando o **projeto final**, treinando **pitch técnico**, estruturando **portfólio no GitHub** e realizando **code review profissional** com **feedback técnico orientado à evolução**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Preparação para o Mercado

◆ O que o mercado realmente avalia?

- ✓ Arquitetura correta
- ✓ Código limpo e organizado
- ✓ Boas práticas (SOLID, Clean Architecture)
- ✓ Segurança básica (JWT, CORS)
- ✓ Capacidade de explicar decisões técnicas

Importante:

O projeto é menos importante do que **como você explica o projeto**.

2 Pitch Técnico

◆ O que é?

Apresentação curta, objetiva e técnica do projeto.

◆ Estrutura ideal (5–7 minutos)

1. Problema

- Qual dor o sistema resolve?

2. Solução

- O que foi construído?

3. Arquitetura

- Stack e decisões técnicas

4. Demonstração

- Fluxo principal funcionando

5. Diferenciais

- Segurança, Docker, Clean Architecture

6. Evolução

- Próximos passos
-

3 Portfólio GitHub Profissional

◆ Estrutura recomendada

📁 projeto-fullstack |— README.md |— src | |— api | |— frontend |— docker-compose.yml |— .github/workflows |— docs

📄 README.md (Modelo Profissional)

Projeto Full Stack — .NET + React ## 📌 Descrição Sistema de gestão de produtos com autenticação JWT, dashboard e deploy via Docker. ## 🛠 Stack - ASP.NET Core 8 - React + Vite - JWT - Docker / Docker Compose ## 🏗 Arquitetura - Clean Architecture - DTOs - Services - Repositories ## 🚀 Execução docker compose up --build ## 🔒 Segurança - Autenticação JWT - Rotas protegidas - CORS configurado ## 📅 Próximos Passos - CI/CD completo - Observabilidade - Kubernetes

📌 README pesa muito em entrevistas.

4 Code Review Profissional

◆ O que será avaliado?

Critério	Avaliação
Organização	Estrutura clara
Legibilidade	Código limpo
Arquitetura	Separação correta
Segurança	JWT, validações
DevOps	Docker funcional

◆ Tipos de feedback

- ✅ Aprovado
- ⚠ Ajustes recomendados
- ❌ Problemas críticos

📌 Mercado não procura código perfeito, procura código sustentável.

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Apresentar o **Projeto Final**, realizar **demonstração técnica**, passar por **code review** e receber **feedback estruturado**.

1 Apresentação do Projeto (Aluno)

Checklist de apresentação:

- ✓ Contexto do problema
- ✓ Arquitetura (API + Front)
- ✓ Demonstração funcionando
- ✓ Segurança (login / rotas)
- ✓ Docker em execução
- ✓ Versionamento

Tempo sugerido:

- 10–12 minutos por aluno/grupo
-

2 Demonstração Técnica

Fluxo mínimo esperado:

1. `docker compose up`
2. Acesso ao frontend
3. Login JWT
4. CRUD funcional
5. Dashboard / listagem
6. Health check da API

🚀 Nada de slides longos — foco no sistema rodando.

3 Code Review (Instrutor)

Checklist técnico:

- Clean Architecture aplicada
 - Controllers enxutos
 - Services com regras
 - DTOs corretos
 - Logs estruturados
 - Docker funcional
 - README claro
-

4 Feedback Técnico Estruturado

Modelo de feedback:

✓ Pontos Fortes

- Arquitetura bem definida
- Boa separação de camadas
- Uso correto de JWT

⚠ Pontos de Melhoria

- Falta tratamento de erro global
- Falta testes automatizados
- Melhorar UX em formulários

🚀 Próximos Passos Recomendados

- Testes com xUnit/Jest
 - CI/CD completo
 - Observabilidade (OpenTelemetry)
 - Cloud (Azure/AWS)
-

✅ Resultado Esperado

- ✓ Projeto Full Stack funcional
 - ✓ Apresentação técnica segura
 - ✓ Portfólio pronto para GitHub/LinkedIn
 - ✓ Feedback claro para evolução
 - ✓ Preparação real para mercado
-

🏁 ENCERRAMENTO DO CURSO

Ao final do curso, o aluno é capaz de:

- ✓ Desenvolver sistemas **Full Stack profissionais**
- ✓ Trabalhar com **.NET, React ou Blazor**
- ✓ Aplicar **Clean Architecture e SOLID**
- ✓ Implementar **JWT e segurança básica**
- ✓ Dockerizar aplicações
- ✓ Apresentar projetos com confiança